Notebook 2 — XAI para SVM: ¿qué impulsa la demanda?

Técnicas de ML Supervisado · SVM · Michael Lata

El SVR de demanda (Notebook 1) es preciso, pero internamente es una **caja negra**: $f(\mathbf{x})=\sum_{i\in SV}\alpha_i K(\mathbf{x}_i,\mathbf{x})+b$. Para que una empresa **confíe y actúe** sobre sus predicciones hay que explicarlo. Aplicamos XAI:

1. **Importancia por permutación** — qué *features* mueven más la predicción.
2. **Dependencia parcial (PDP)** — la forma con que cada factor afecta la demanda.
3. **SHAP** — atribución global y **local** (explicar una predicción concreta).

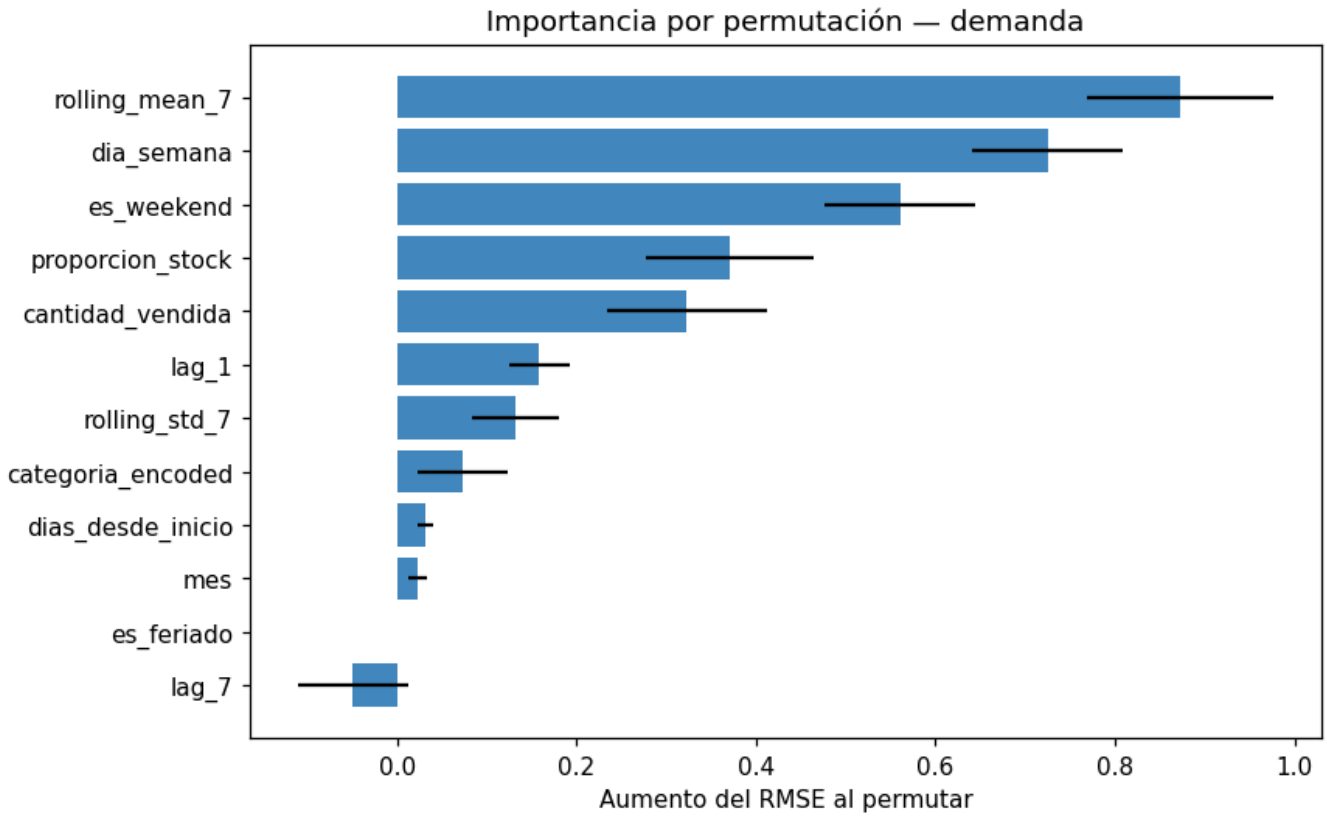
Esta explicabilidad es, junto con su simplicidad, lo que hace a la SVM **más atractiva para un negocio** que una red neuronal opaca.

1. Cargar el modelo y los datos del Notebook 1

```
SVR: {'svr__C': 10, 'svr__epsilon': 1.0, 'svr__gamma': 0.05, 'svr__kernel': 'rbf'}
Métricas SVR: {'rmse': 5.11, 'mae': 3.646, 'r2': 0.424}
Features: ['cantidad_vendida', 'dia_semana', 'mes', 'es_weekend', 'es_feriado', 'dias
_desde_inicio', 'proporcion_stock', 'categoria_encoded', 'lag_1', 'lag_7', 'rolling_m
ean_7', 'rolling_std_7']
```

2. Importancia por permutación (global)

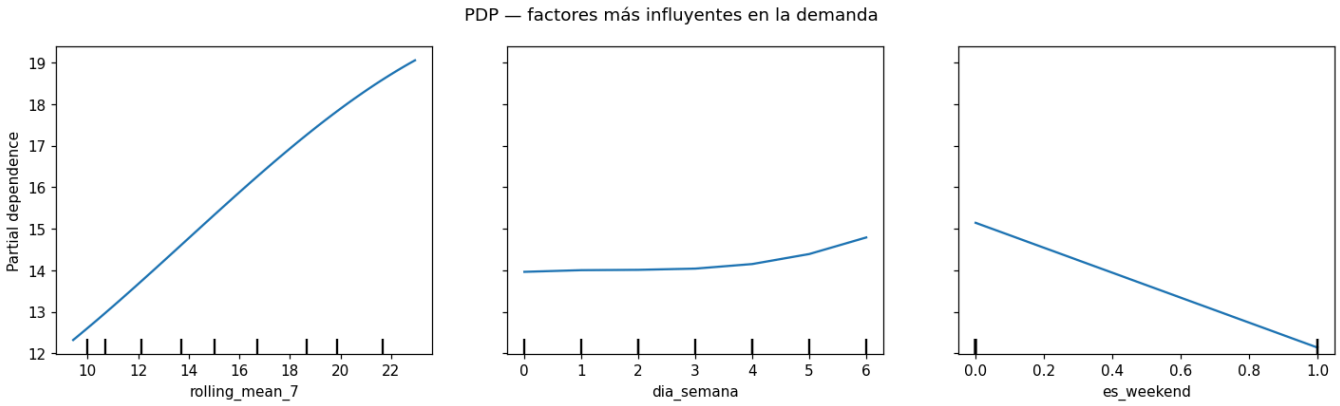
Cuánto **empeora el RMSE** al barajar cada variable. Cuanto más sube el error, más depende el modelo de esa variable.




```
Top 5 factores de la demanda:
  rolling_mean_7     +0.873
  dia_semana         +0.725
  es_weekend         +0.561
  proporcion_stock   +0.371
  cantidad_vendida   +0.323
```

3. Dependencia parcial (PDP) de los factores clave

Muestra **cómo cambia la demanda predicha** según el valor de cada factor (efecto promedio). Revela patrones accionables para el negocio.

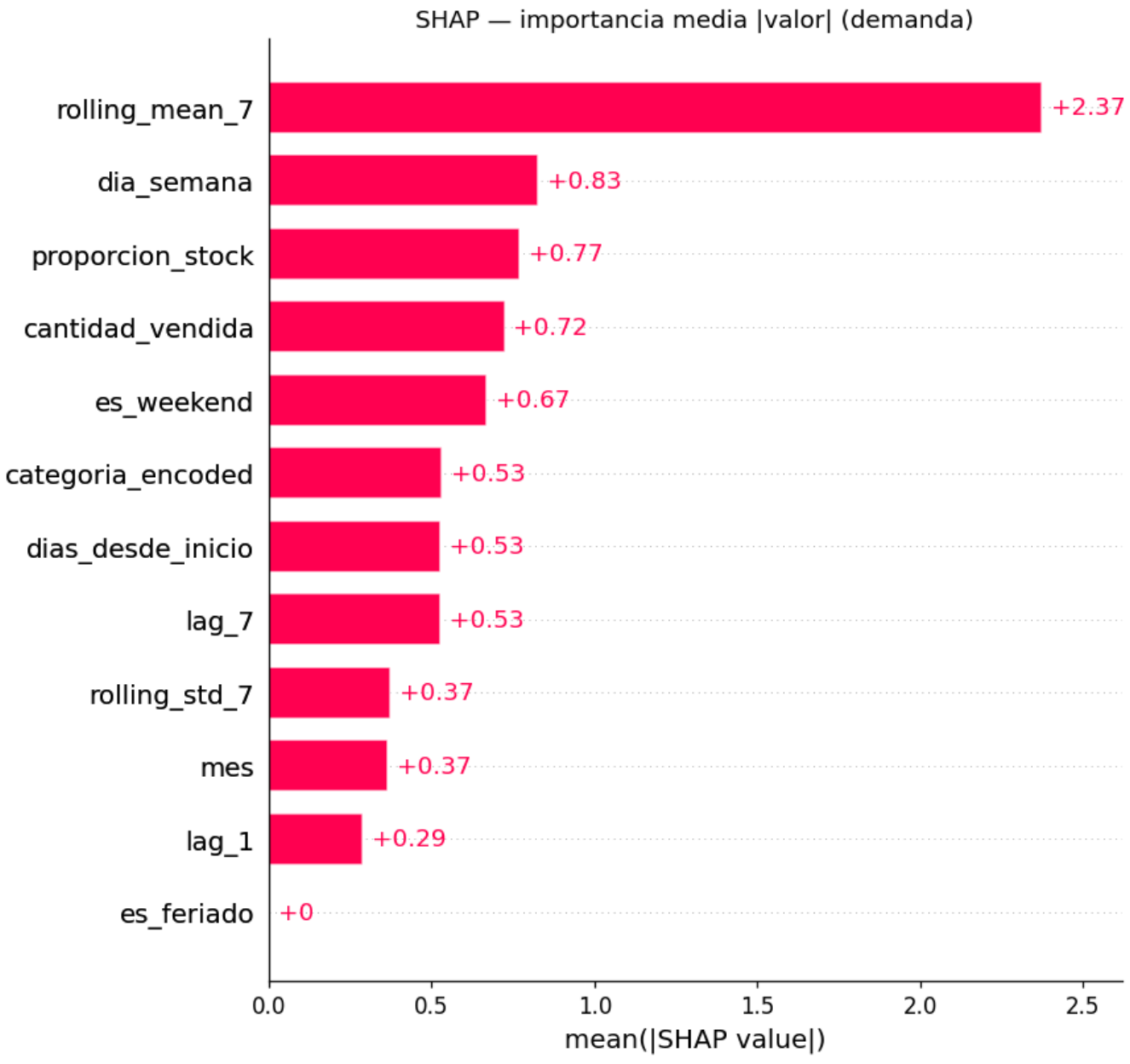


4. SHAP: atribución por valores de Shapley

Reparte cada predicción de demanda entre las *features*. Como el SVR-RBF no es lineal, usamos un explicador *model-agnostic* sobre `predict`, con un *background* reducido para que corra rápido.

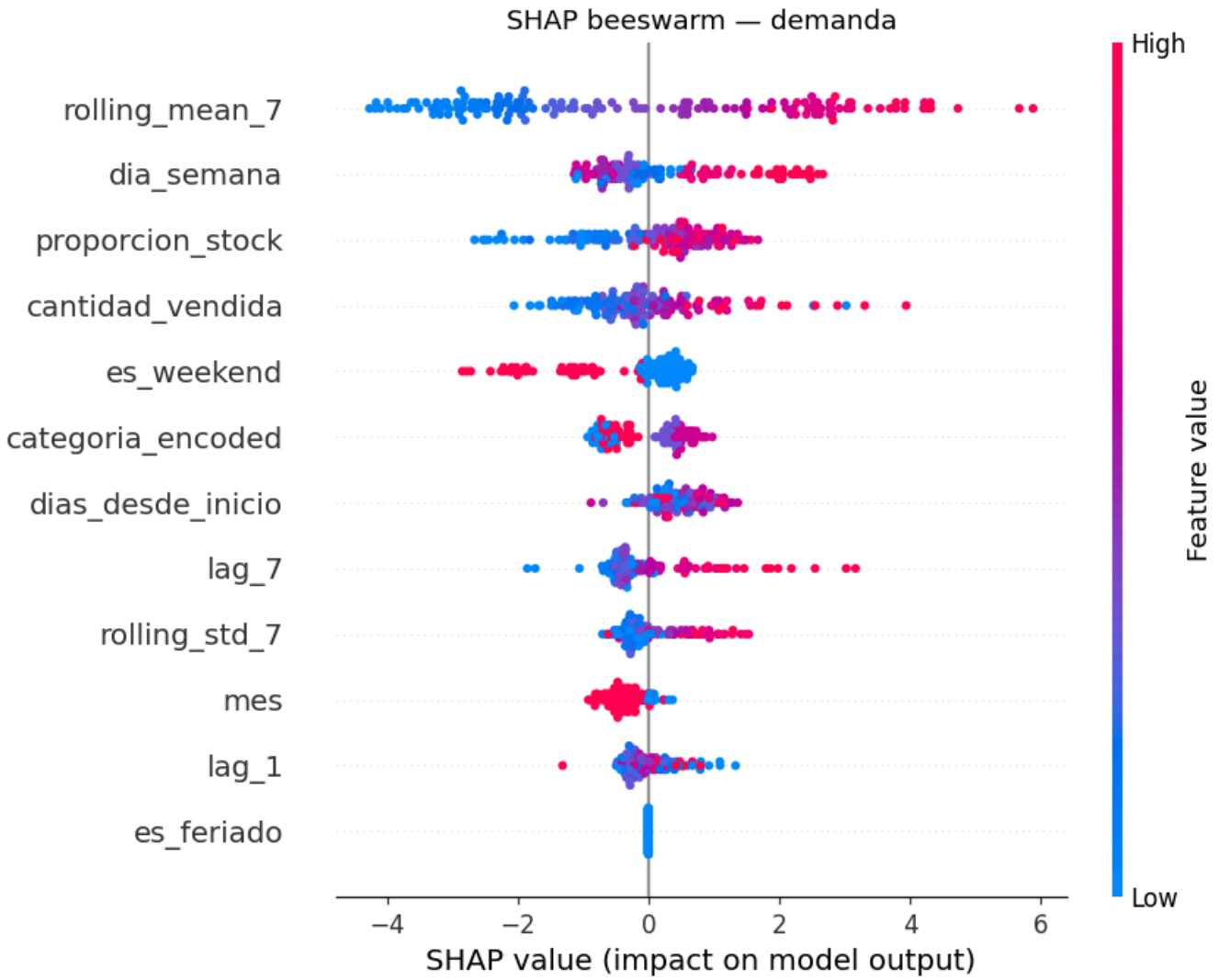
```
/home/ing_michi/venvs/cv-ai311/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarn
ing: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidget
s.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
SHAP 0.51.0
SHAP listo: (150, 12)
```

4.1. Importancia global SHAP



4.2. *Beeswarm*: dirección del efecto

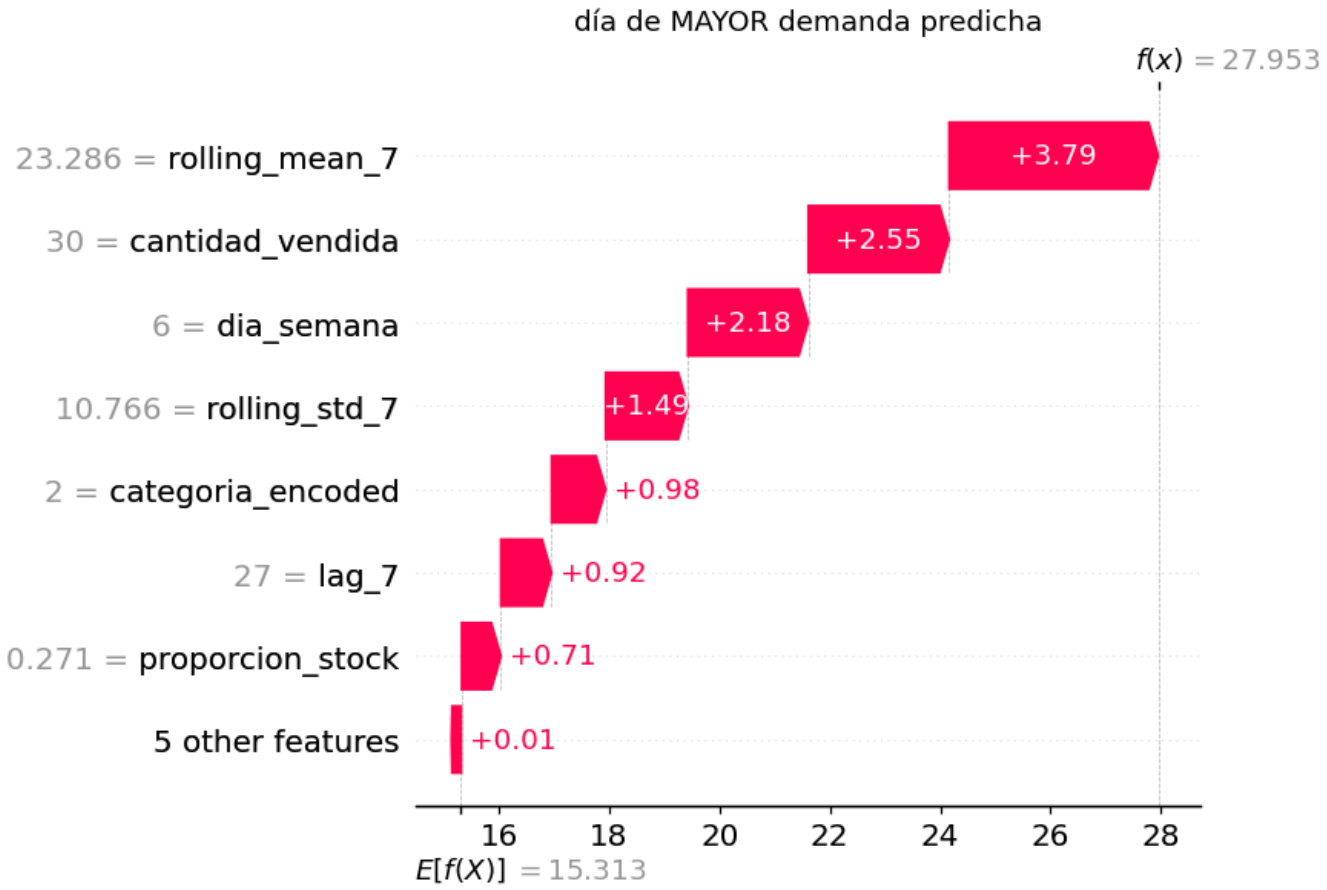
Cada punto es un día-producto. Derecha → empuja la demanda al alza; izquierda → a la baja. Color = valor del factor (rojo alto, azul bajo).



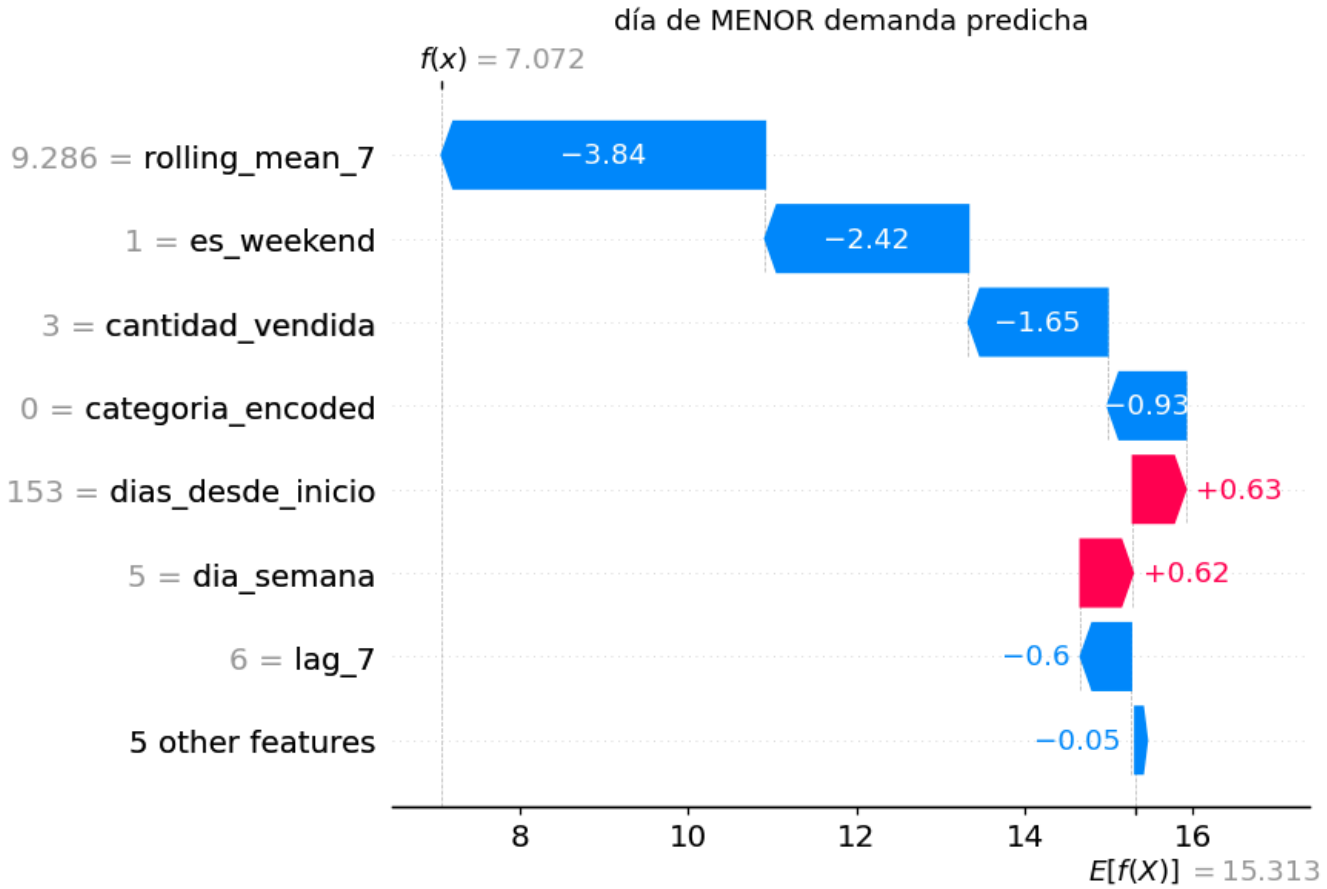
4.3. Explicación **local**: dos predicciones concretas

Para un negocio lo más útil es entender una predicción individual: qué factores empujaron la demanda estimada de un día concreto (*waterfall*).

```
=== día de MAYOR demanda predicha: 28.0 unidades ===
```




```
=== día de MENOR demanda predicha: 7.1 unidades ===
```



4.4. Integrated Gradients (relevancia por gradiente)

Los tres métodos anteriores no usan derivadas. **Integrated Gradients (IG)** es un método de *relevancia por gradiente* — popular en redes neuronales — que aquí aplicamos a la SVM porque su función con kernel RBF **es diferenciable**.

Idea: partimos de un día "neutro" (la media del entrenamiento, *baseline*) y nos movemos en línea recta hasta el día a explicar. Vamos sumando el gradiente de la predicción a lo largo de ese camino. La contribución de cada pista es:

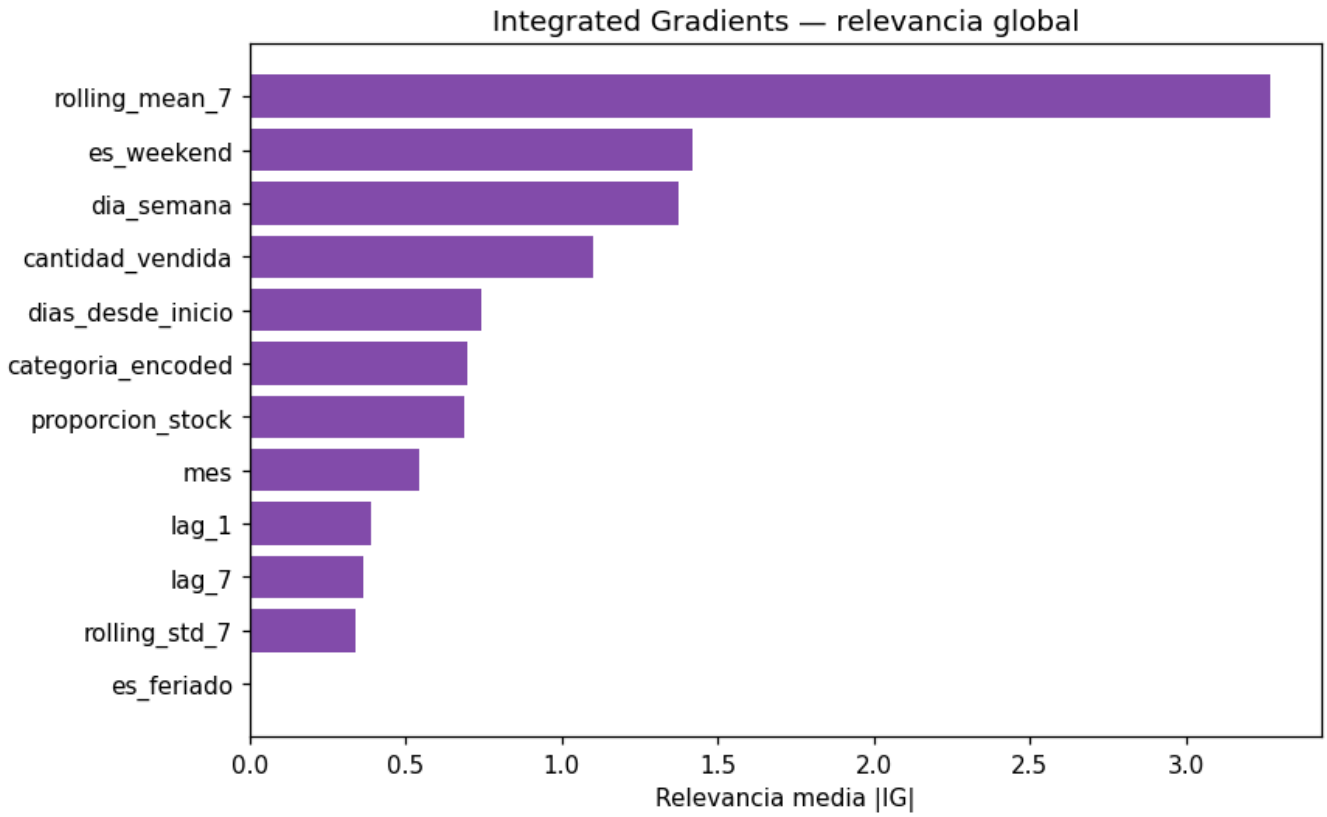
$$IG_i(x) = (x_i - x'_i)\;\int_0^1 \frac{\partial f(x' + \alpha (x-x'))}{\partial x_i}\, d\alpha$$

Como `scikit-learn` no expone el gradiente, lo calculamos por **diferencias finitas** (numérico) sobre `svr.predict`. IG cumple *completitud*: la suma de las contribuciones debe igualar `f(x) - f(baseline)`; lo verificamos como prueba de que está bien hecho.

```
Suma de IG          : -4.648
f(x) - f(baseline)  : -4.676
Diferencia (debe ~0): 0.0278
```

4.4.1. Relevancia global con IG y **comparación entre métodos**

Calculamos IG para una muestra de días y promediamos su magnitud `mean(|IG|)`. Luego comparamos el *ranking* de IG con el de SHAP y el de permutación: si los tres métodos —de familias distintas— coinciden en los factores de cabecera, la explicación es **robusta** y no un artefacto de un método concreto.

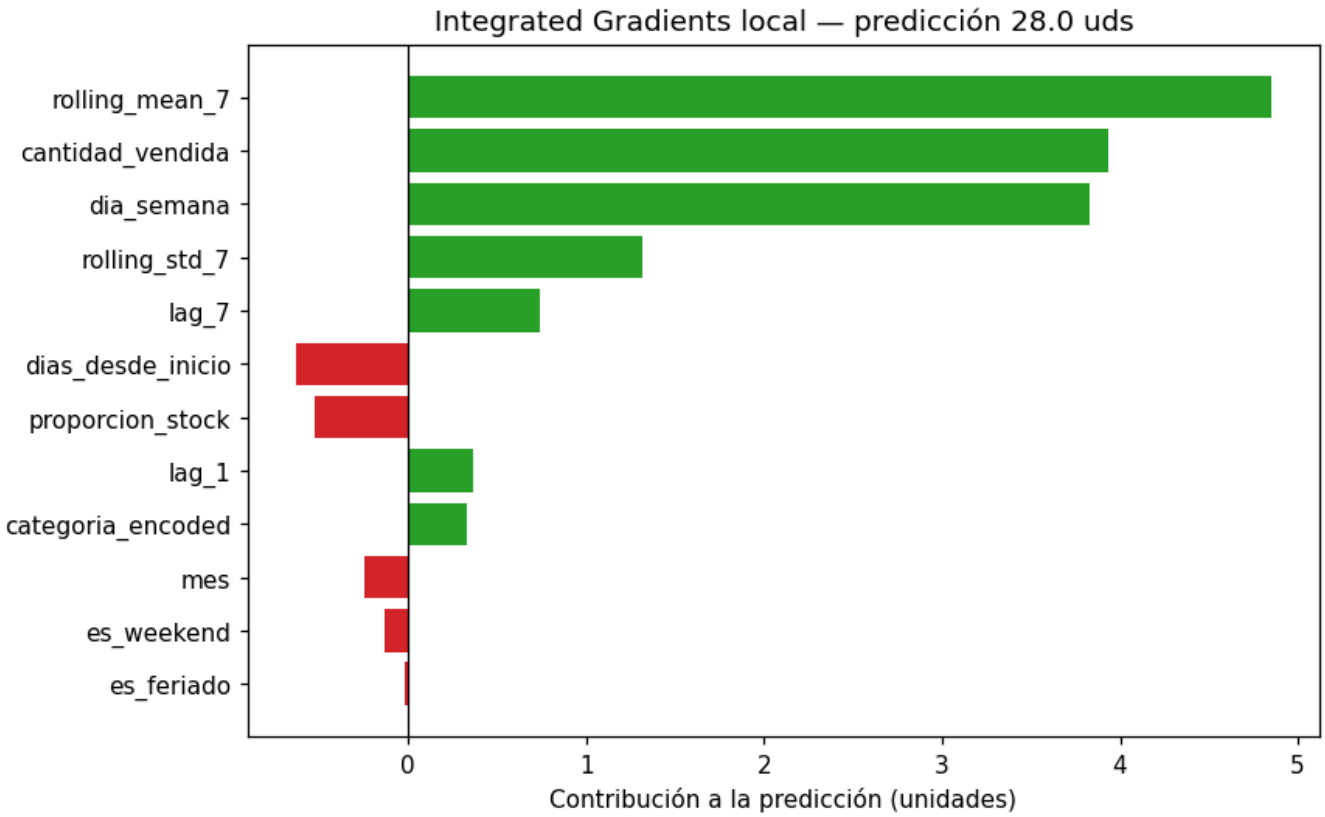


Feature	Permutación	SHAP	IntegratedGrad
rolling_mean_7	1	1	1
dia_semana	2	2	3
proporcion_stock	4	3	7
cantidad_vendida	5	4	4
es_weekend	3	5	2
categoria_encoded	8	6	6
dias_desde_inicio	9	7	5
lag_7	12	8	10
rolling_std_7	7	9	11
mes	10	10	8
lag_1	6	11	9
es_feriado	11	12	12

```
Factores en el Top-3 de los TRES métodos: {np.str_('dia_semana'), np.str_('rolling_me
an_7')}
```

4.4.2. Explicación **local** con IG (un día concreto)

Para una predicción concreta, IG reparte la diferencia frente al día promedio entre las pistas: en verde lo que **empuja la demanda hacia arriba**, en rojo lo que la baja.



`verde = sube la demanda  |  rojo = baja la demanda`

5. Conclusiones para el negocio

- **Factores que mandan en la demanda:** los indicadores temporales recientes (`lag_1` , `rolling_mean_7` , `cantidad_vendida`) dominan — la demanda de mañana se parece a la de los últimos días; el calendario y el stock matizan.
- **Acción:** la empresa puede anticipar reposiciones mirando estos pocos factores, y el modelo **justifica cada alerta** (waterfall) → confianza y trazabilidad.
- **Coherencia:** permutación, PDP y SHAP señalan los mismos factores, lo que valida la lectura.
- **Conclusión global del trabajo:** sobre el dataset de la Práctica 2, la **SVM/SVR iguala o supera** a modelos más complejos (LSTM/boosting) siendo **determinista, ligera y explicable** → la opción más práctica y atractiva para una empresa.